

## DATA STRUCTURES AND ALGORITHMS

### UNIT 3 PROJECT REPORT

# Bus Route Stops Using Doubly Linked List

|                             |                                          |
|-----------------------------|------------------------------------------|
| <b>Project</b>              | Bus Route Stops Using Doubly Linked List |
| <b>Programming Language</b> | C++                                      |
| <b>Data Structure</b>       | Doubly Linked List                       |
| <b>Student</b>              | Aditya Barate                            |
| <b>Student ID</b>           | 109                                      |

A C++ programming assignment demonstrating linked-list operations in a practical bus-route scenario.

## 1. Introduction

A linked list is a dynamic data structure made up of nodes connected through pointers. This project uses a Doubly Linked List to represent a bus route. Each node stores one stop name and links to the preceding and following stops.

## 2. Problem Statement

Develop a C++ program to manage bus route stops using a Doubly Linked List. Each node stores the name of a bus stop and pointers to the previous and next stops. The program supports adding and removing stops, displaying the route in forward and backward directions, and finding the previous and next stops for a selected stop.

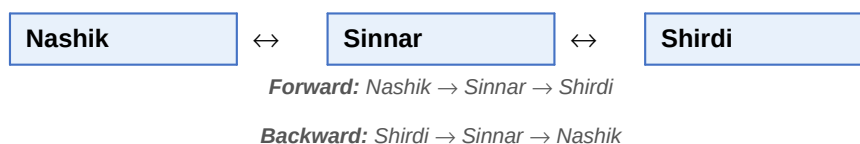
## 3. Objectives

- Understand the structure and operation of a Doubly Linked List.
- Implement insertion and deletion of nodes.
- Traverse the route in both directions.
- Find a stop's previous and next stops.
- Apply a fundamental data structure to a practical scenario.

## 4. Theory: Doubly Linked List

Each node contains three fields: **prev** (pointer to the previous node), **data** (the bus stop name), and **next** (pointer to the next node). The program maintains **head** at the first node and **tail** at the last node. The next links enable forward traversal and the prev links enable backward traversal. The first node's prev and the last node's next are null.

Example route:



## 5. Functional Requirements

- **Add Stop:** Create a node and append it to the route.
- **Remove Stop:** Find the stop, reconnect neighboring nodes, update head or tail if needed, and delete the node.
- **Display Forward Route:** Traverse from head by following next pointers.
- **Display Return Journey:** Traverse from tail by following prev pointers.
- **Find Previous and Next:** Search for a stop and display adjacent stops, or indicate if it is the first or last stop.

## 6. Algorithm

### Main menu

- Initialize an empty list with head and tail set to null.
- Display the menu and read the user's choice.
- Call the function for the selected operation.
- Repeat until the user selects Exit.

### Insertion at the end

- Create a node and store the stop name.
- Set the new node's next pointer to null and prev pointer to tail.
- If the list is empty, set head and tail to the new node.
- Otherwise, connect the old tail to the new node and update tail.

### Deletion by stop name

- Search from head until the stop is found or the list ends.
- If it is not found, display a message.
- Reconnect the previous and next nodes where they exist.
- Update head or tail when removing an endpoint, then delete the node.

## 7. System Design

The application is menu-driven. The linked list stores stops in travel order. The head and tail pointers make it possible to traverse from either end. Each operation is handled by a separate function.

## 8. Complexity Analysis

| Operation           | Time   | Reason                                        |
|---------------------|--------|-----------------------------------------------|
| Add at end          | $O(1)$ | Tail provides direct access to the last node. |
| Remove by name      | $O(n)$ | Searching may visit every node.               |
| Forward display     | $O(n)$ | Visits each node once.                        |
| Backward display    | $O(n)$ | Visits each node once.                        |
| Find adjacent stops | $O(n)$ | Searches for the requested stop.              |

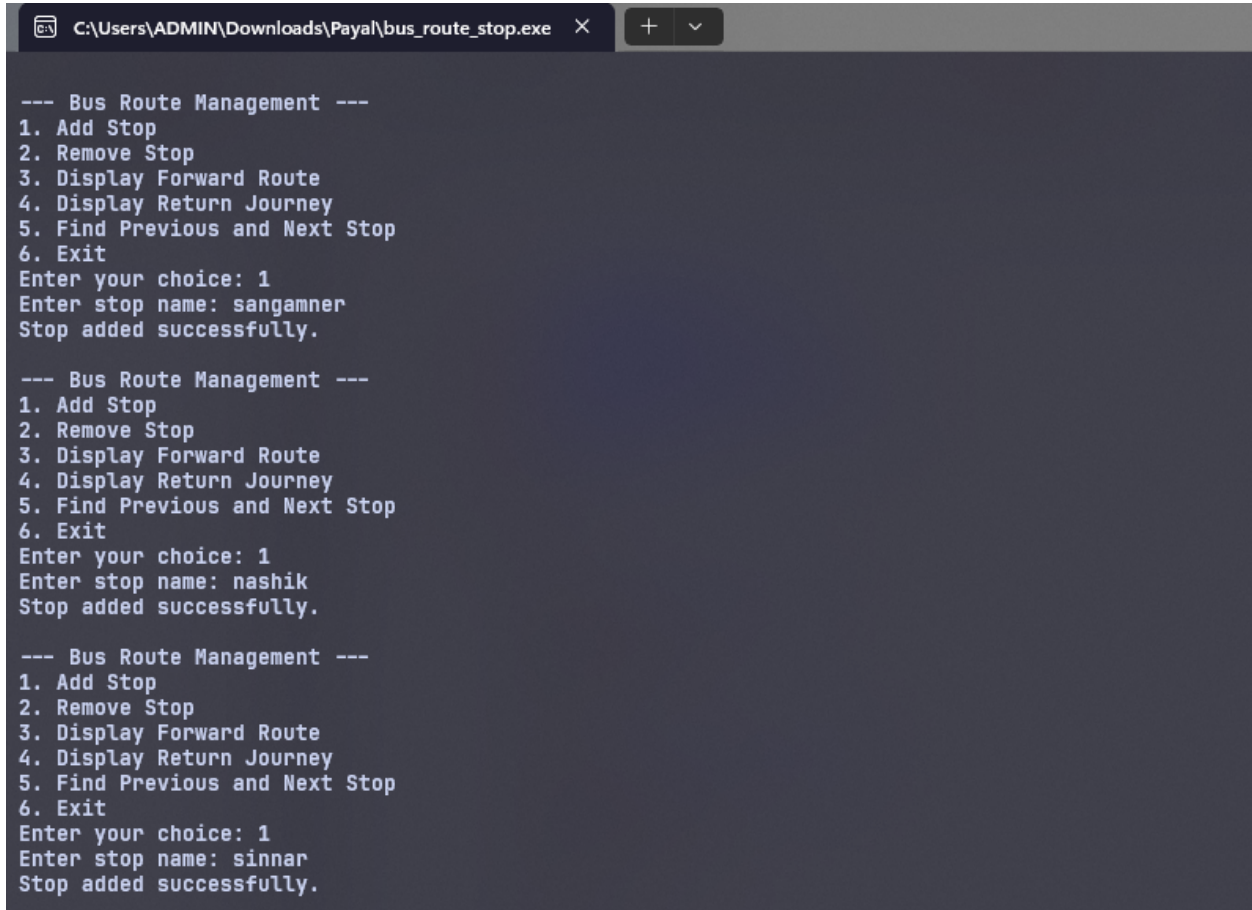
The list requires  $O(n)$  space for  $n$  stop nodes.

## 9. Testing and Output Evidence

The following screenshots were provided as evidence of the program execution. They demonstrate insertion, forward and backward traversal, adjacent-stop lookup, and deletion.

### Screenshot 1 — Adding bus stops

The program successfully adds Sangamner, Nashik, and Sinnar.



```
--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 1
Enter stop name: sangamner
Stop added successfully.

--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 1
Enter stop name: nashik
Stop added successfully.

--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 1
Enter stop name: sinnar
Stop added successfully.
```

Figure 1: Program output screenshot supplied for this report.

## Screenshot 2 — Route traversal and lookup

The forward route, return journey, and previous/next stops for Nashik are displayed.

```
--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 3
Forward Route: sangamner → nashik → sinnar

--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 4
Return Journey: sinnar → nashik → sangamner

--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 5
Enter current stop: nashik
Previous Stop: sangamner
Next Stop: sinnar
```

Figure 2: Program output screenshot supplied for this report.

### Screenshot 3 — Removing a stop

Sinnar is removed successfully and the forward route is updated.

```
--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 2
Enter stop to remove: sinnar
Stop removed successfully.

--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: 3
Forward Route: sangamner → nashik

--- Bus Route Management ---
1. Add Stop
2. Remove Stop
3. Display Forward Route
4. Display Return Journey
5. Find Previous and Next Stop
6. Exit
Enter your choice: █
```

Figure 3: Program output screenshot supplied for this report.

## 10. Observations and Limitations

The screenshots show that the program adds stops, displays the route in both directions, finds adjacent stops, and removes a stop while updating the route. The sample route is Sangamner → Nashik → Sinnar; after Sinnar is removed, the forward route becomes Sangamner → Nashik.

- Stop lookup and deletion depend on the entered stop name matching the stored text.
- If duplicate names are allowed, searching by name may find the first matching node.
- The route is stored in memory and is not preserved after the program exits.
- The program models stop order only; it does not calculate distances, travel times, or live bus information.

## 11. Conclusion

This project applies a Doubly Linked List to a bus-route management problem. It demonstrates insertion, deletion, forward and backward traversal, and finding adjacent stops, providing practical experience with pointers and dynamic memory allocation.

## 12. Repository Structure

```
DSA-Unit3-LinkedList-109-BarateAditya/  
■■■ README.md  
■■■ Source_Code/  
■ ■■■ linkedlist.cpp  
■■■ Output/  
■ ■■■ output.pdf  
■ ■■■ screenshots/  
■■■ Documentation/  
■■■ Report.pdf
```

## 13. References

Course notes and classroom material on linked lists and C++ data structures. Verify the report against the final submitted source code before submission.